Pipes in the Desert
Analysis Model - Version 1

E5 - TEAM A

Supervisor:

Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

March 8th, 2026

3. Analysis model – version 1

3.0. Corrections

3.0.1 References (2.1.3)

[5] OpenAI, “ChatGPT.” Used for proofreading, grammar and punctuation corrections, brainstorming ideas, and helping to understand potential edge cases during the preparation of the document.

3.0.2 Functional Requirements (2.3)

ID	Description	Check	Priority	Source	Use-case
FR-01	The system shall allow the players to enter player names before starting a game session.	Start is unavailable until all player names are entered.	Must have	Group decision	Start / Supervise Game
FR-02	The system shall set the duration of the game.	Start is unavailable until duration is set.	Must have	Group decision	Start / Supervise Game
FR-03	The system shall require at least two Plumbers and at least two Saboteurs before a game session can start.	The system refuses to start if either team has fewer than two players.	Must have	Problem definition	Start / Supervise Game
FR-04	The system shall generate an initial network containing at least one spring, one cistern, one pump, and connecting pipes when a new game starts.	After pressing start, the initial network is visible and contains all mandatory element types.	Must have	Problem definition	Start / Supervise Game
FR-05	The system shall start the game session only after all start conditions are satisfied.	The game begins only if player data, team sizes, and duration are valid.	Must have	Professor feedback	Start / Supervise Game
FR-06	The system shall display the full pipe network, player positions, scores, and remaining time during the running game.	All listed elements are visible and updated during play.	Must have	Current document/feedback alignment	Start / Supervise Game

FR-07	Water shall flow from springs through the network during the game.	Water advances from springs during active turns.	Must have	Problem definition	Start / Supervise Game
FR-08	Water that reaches a cistern shall be counted as delivered water.	The delivered water counter increases when water reaches a cistern.	Must have	Problem definition	Start / Supervise Game
FR-09	Water that reaches a free end pipe end shall be counted as lost water.	The lost-water counter increases when water reaches a free end.	Must have	Problem definition	Start / Supervise Game
FR-10	A pump shall have exactly one active input pipe and one active output pipe at a time.	At any time, only one input and one output are active for a pump.	Must have	Problem definition	Change Pump Direction
FR-11	Water shall pass through a pump only between its active input and active output pipes.	Water flows only along the currently selected route.	Must have	Problem definition	Change Pump Direction
FR-12	A broken pump shall not transfer water.	No water passes through a broken pump.	Must have	Problem definition	Randomly Break Pump
FR-13	A pump shall not transfer water if its active output pipe has no free capacity.	Water transfer stops when the output pipe is full.	Must have	Problem definition	Start / Supervise Game
FR-14	The system shall prevent connecting more pipes to a pump than its connection limit allows.	Connection attempt beyond the limit is refused.	Must have	Problem definition	Start / Supervise Game

FR-15	A Player shall be able to move only along pipes and pumps.	Movement outside the network is rejected.	Must have	Problem definition	Move Player
FR-16	A Player shall be able to move only to an adjacent connected element. (pipes, pumps, cisterns, spring)	Non-adjacent movement is rejected.	Must have	Group decision based on game logic	Move Player
FR-17	At most one Player may occupy a pipe at a time.	A move to an already occupied pipe is rejected.	Must have	Problem definition	Move Player
FR-18	Multiple Players may occupy the same pump at the same time.	Several players can stand on one pump.	Must have	Problem definition	Move Player
FR-19	The system shall enforce turn-based play.	Only the current player can act.	Must have	Problem definition	Move Player
FR-20	The system shall allow at most one action for a player during one turn. A plumber can not move and pick up things at the same turn.	A second action in the same turn is rejected.	Must have	Group decision / previous FR clarificatio n	Move Player
FR-21	A Plumber shall be able to repair a broken pump when standing on that pump.	Repair succeeds only if the player is a Plumber and is on the broken pump.	Must have	Problem definition	Repair Pipe/Pump
FR-22	A repaired pump shall resume normal water transfer.	Water can again pass through the repaired pump.	Must have	Problem definition	Repair Pipe/Pump

FR-23	A Plumber shall be able to repair a punctured pipe when standing on that pipe.	Repair succeeds only if the player is a Plumber and is on the punctured pipe.	Must have	Problem definition	Repair Pipe/Pump
FR-24	A repaired pipe shall stop leaking water.	Water loss at that pipe stops after repair.	Must have	Problem definition	Repair Pipe/Pump
FR-25	A Plumber shall be able to change the active direction of a pump when standing on that pump.	Pump direction changes only when a Plumber stands on the target pump.	Must have	Problem definition	Change Pump Direction
FR-26	A Saboteur shall be able to change the active direction of a pump when standing on that pump.	Pump direction changes only when a Saboteur stands on the target pump.	Must have	Problem definition	Change Pump Direction
FR-27	A pipe generated at a cistern shall have one end connected to the cistern and the other end free.	After generation, the new pipe has exactly one end connected to the cistern and the other end not connected to any element.	Must have	Problem definition	Place New Pipe
FR-28	A Plumber shall be able to connect the free end of a pipe to an adjacent network element while standing on that pipe end.	When a Plumber stands at the free end of a pipe and performs the connect action, the pipe becomes connected to the selected adjacent element.	Must have	Problem definition	Place New Pipe
FR-29	A Plumber shall be able to collect a new pump generated at a cistern.	Pump can be collected only if one is available at the cistern.	Must have	Problem definition	Insert New Pump
FR-30	A Plumber shall be able to insert a collected pump into an existing pipe by splitting that pipe and connecting both new ends to the pump.	After insertion, the original pipe is replaced by two pipe segments and one pump.	Must have	Problem definition	Insert New Pump

FR-31	A Saboteur shall be able to puncture a pipe when standing on that pipe.	Puncture succeeds only if the player is a Saboteur and is on the target pipe.	Must have	Problem definition	Puncture Pipe
FR-32	A punctured pipe shall leak water to the desert until repaired.	Water loss continues on that pipe until repair occurs.	Must have	Problem definition	Puncture Pipe
FR-33	The system shall randomly select pumps to break during the game.	During play, random pump breakdown events occur.	Must have	Problem definition	Randomly Break Pump
FR-34	A randomly broken pump shall remain broken until repaired by a Plumber.	The pump stays non-functional until repair.	Must have	Problem definition	Randomly Break Pump
FR-35	The system shall periodically generate new pipes at cisterns during the game.	New pipes appear at cisterns after the defined interval.	Must have	Problem definition	Generate New Pipes / Pumps
FR-36	The system shall periodically generate new pumps at cisterns during the game.	New pumps appear at cisterns after the defined interval.	Must have	Problem definition	Generate New Pipes / Pumps
FR-37	The system shall end the game when the game timer reaches zero.	When remaining time becomes zero, gameplay stops.	Must have	Professor feedback / current rules	End Game / Declare Winner
FR-38	When the game ends, the system shall compare total delivered water and total lost water.	Final comparison is shown on the result screen.	Must have	Problem definition	End Game / Declare Winner

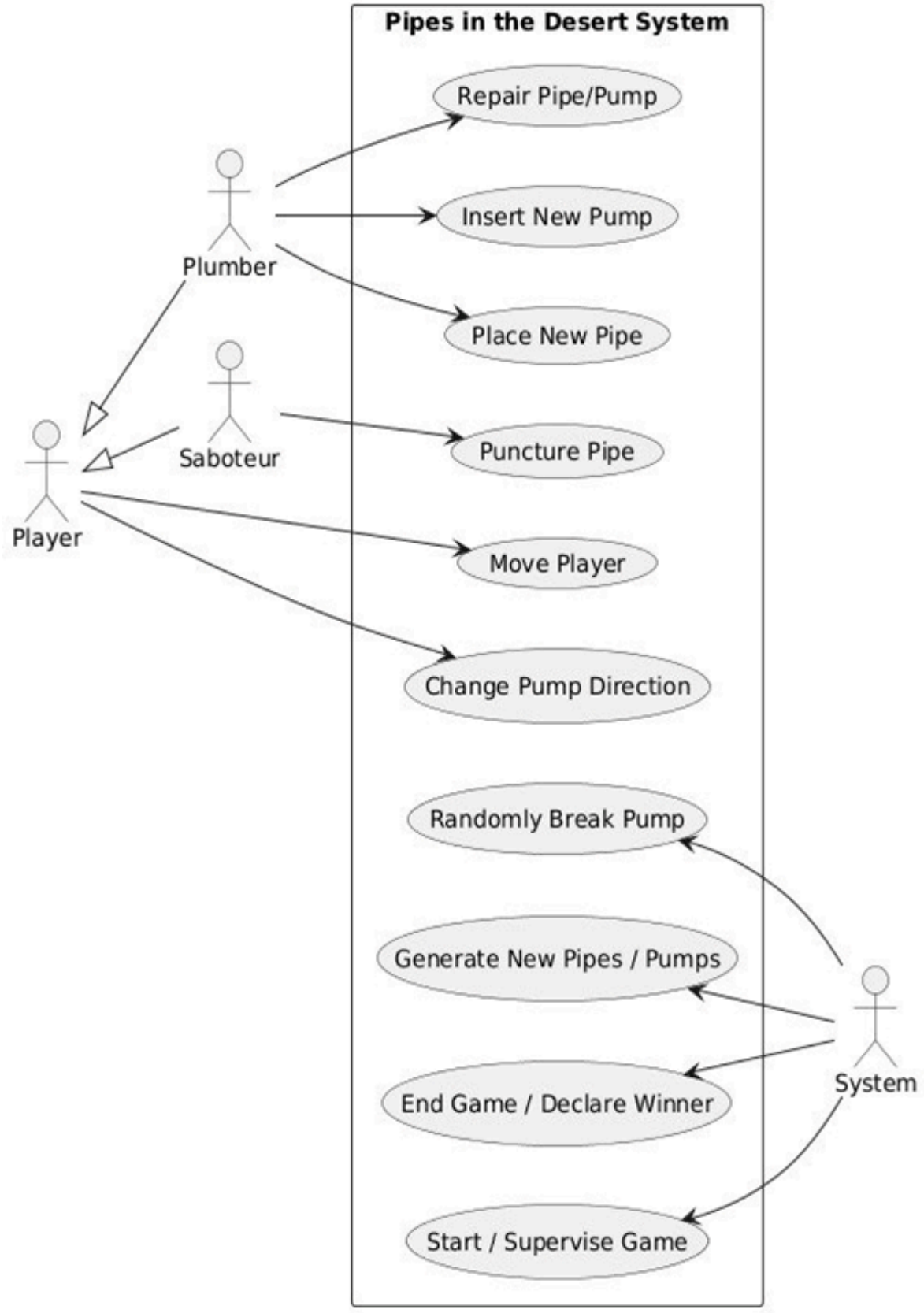
FR-39	When the game ends, the Plumber team shall win if delivered water is greater than lost water; otherwise the Saboteur team shall win.	The displayed winner matches the final score rule.	Must have	Problem definition	End Game / Declare Winner
FR-40	When the game ends, the system shall display the final result and the winning team.	Final result screen appears after the timer expires.	Must have	Professor feedback / current document	End Game / Declare Winner

3.0.2 Non - Functional Requirements (2.3.3)

ID	Description	Check	Priority	Source	Comment
NF-01	A new player shall be able to understand the basic rules and start playing without external help within 10 minutes.	A new user can start and play one full round after at most 10 minutes of familiarization.	Must have	Group Meeting	ISO 25010 – Usability
NF-02	The system shall not crash or produce unhandled errors during normal gameplay.	No unhandled exception occurs during one full game session.	Must have	Group Meeting	ISO 25010 – Reliability
NF-03	If an unexpected internal error occurs, the system shall display a user-friendly error message instead of terminating abruptly.	When an internal error is triggered, an understandable error message is shown and the application does not close unexpectedly.	Must have	Group Meeting	Split from old NF-02; ISO 25010 – Reliability
NF-04	The game shall respond to a player action within 1 second on the minimum required hardware.	Measured response time from player action to system reaction is at most 1 second.	Should have	Group Meeting	Split from old NF-03; ISO 25010 – Performance efficiency
NF-05	The graphical user interface shall visibly update within 1 second after each completed action on the minimum required hardware.	Screen state changes become visible within 1 second after each action.	Should have	Group Meeting	Split from old NF-03; ISO 25010 – Performance efficiency

NF-06	The source code shall follow a consistent coding style and be structured into logical modules so that core components can be tested independently.	Unit tests can be written for core game logic without modifying production code.	Must have	Group Meeting	Old NF-04 renumbered; ISO 25010 – Maintainability
NF-07	The system shall run on any operating system that supports Java JDK 17 without requiring source-code changes.	The application compiles and runs on Windows and at least one other Java JDK 17-compatible operating system.	Must have	Group Meeting	Old NF-05 renumbered; ISO 25010 – Portability

3.0.3 Use-Case Diagram



3.0.4 Use-Case Descriptions

Start / Supervise Game

Use-case name	Start / Supervise Game
Actors	System (secondary)
Related FRs	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, FR-09, FR-13, FR-14
Preconditions	No game session is currently running. Player names, team sizes, and game duration are known.
Dialog / Scenario	1. System prompts for player names and team assignments; start is unavailable until all fields are filled (FR-01). 2. System requires a game duration to be set before starting (FR-02). 3. System verifies at least two Plumbers and two Saboteurs are assigned; refuses to start otherwise (FR-03). 4. System generates an initial network containing at least one spring, one cistern, one pump, and connecting pipes (FR-04). 5. System starts the game only after all start conditions are satisfied (FR-05). 6. System displays the full pipe network, player positions, scores, and remaining time throughout the game (FR-06). 7. Water flows from springs through the network; water reaching a cistern is counted as delivered (FR-07, FR-08); water reaching a free pipe end is counted as lost (FR-09). 8. Water transfer through a pump stops when the output pipe has no free capacity (FR-13). 9. When the timer expires, End Game / Declare Winner is triggered automatically. 10. The system will be supervising the limits of the pump during the game (FR-14)

Move Player

Use-case name	Move Player
Actors	Player → Plumber (primary), Saboteur (primary)
Related FRs	FR-15, FR-16, FR-17, FR-18, FR-19, FR-20
Preconditions	It is the player's turn. The game session is active.

Dialog / Scenario	1. Player selects an adjacent connected element (pipe, pump, cistern, or spring) as destination (FR-16). 2. System rejects movement outside the network (FR-15). 3. System rejects movement to a pipe already occupied by another player (FR-17). 4. If the destination is a pump, multiple players may occupy it simultaneously (FR-18). 5. System enforces turn-based play; only the current player can act (FR-19). 6. Each player is limited to one action per turn — moving and picking up cannot be combined (FR-20). 7. If valid: System moves the player and updates the display. Turn passes to the next player.

Place New Pipe

Use-case name	Place New Pipe
Actors	Plumber (primary), System (secondary)
Related FRs	FR-27, FR-28
Preconditions	A new pipe component is available at a cistern. The Plumber is located at that cistern.
Dialog / Scenario	1. The System generates a new pipe at a cistern with one end connected to the cistern and the other end free (FR-27). 2. The Plumber moves to the free end of the pipe. 3. The Plumber selects the “Connect Pipe” action. 4. The Plumber selects an adjacent network element as the connection target. 5. The System validates the connection (e.g., connection limits of the target element). 6. If valid, the System connects the free end of the pipe to the selected element and recalculates the water flow; otherwise, the System displays an error and the pipe remains unconnected (FR-28).

Repair Pipe/Pump

Use-case name	Repair Pipe/Pump
Actors	Plumber (primary)
Related FRs	FR-21, FR-22, FR-23, FR-24
Preconditions	The Plumber is located on the target pump or pipe. The element is in a damaged or malfunctioning state.

Dialog / Scenario	1. Plumber selects the "Repair" action on the element they are standing on. 2. System validates that the element is damaged and the actor is a Plumber. 3. If the target is a broken pump: System restores it; water transfer resumes (FR-21, FR-22). 4. If the target is a punctured pipe: System restores it; water loss at that segment stops (FR-23, FR-24). 5. System recalculates water flow and updates the visual indicator.

Insert New Pump

Use-case name	Insert New Pump
Actors	Plumber (primary)
Related FRs	FR-14, FR-29, FR-30
Preconditions	A new pump component is available at a cistern. The Plumber is located at that cistern.
Dialog / Scenario	1. Plumber selects "Pick Up Pump" at the cistern; succeeds only if a pump is available (FR-29). 2. Plumber moves to the target pipe segment. 3. Plumber selects "Insert Pump"; System splits the pipe and places the pump between the two resulting segments (FR-30). 4. System rejects insertion if the pump connection limit would be exceeded (FR-14). 5. System assigns a default inlet/outlet configuration and recalculates water flow.

Change Pump Direction

Use-case name	Change Pump Direction
Actors	Player → Plumber (primary), Saboteur (primary)
Related FRs	FR-10, FR-11, FR-25, FR-26
Preconditions	The player is located on the target pump. The pump is in a working state.
Dialog / Scenario	1. Player selects "Change Pump Direction" on the pump they are standing on. 2. System presents available inlet–outlet routing options; a pump has exactly one active input and one active output at a time (FR-10). 3. Player selects the desired configuration. 4. System updates the pump's active routing; water flows only along the selected route (FR-11). 5. A Plumber uses this to optimise delivery (FR-25); a Saboteur uses it to divert water away from cisterns (FR-26). 6. System recalculates water flow across the network and updates the display.

Puncture Pipe

Use-case name	Puncture Pipe
Actors	Saboteur (primary)
Related FRs	FR-31, FR-32
Preconditions	The Saboteur is located on the target pipe segment. The pipe is currently intact.
Dialog / Scenario	1. Saboteur selects "Puncture Pipe" on the pipe segment they are standing on. 2. System confirms the pipe is intact and the actor is a Saboteur (FR-31). 3. System marks the pipe as damaged; water leaks to the desert and is counted as lost each turn (FR-32). 4. System updates the visual indicator (e.g. red highlight). 5. The pipe remains damaged until a Plumber performs Repair Pipe/Pump.

Randomly Break Pump

Use-case name	Randomly Break Pump
Actors	System (secondary)
Related FRs	FR-12, FR-33, FR-34
Preconditions	The game session is active. At least one pump is in a working state.
Dialog / Scenario	1. System timer triggers a random pump-breakdown event during the game (FR-33). 2. System selects a working pump at random. 3. System sets the pump state to "malfunctioning"; the broken pump does not transfer water (FR-12). 4. System updates the visual indicator on the pump. 5. The pump remains broken until a Plumber performs Repair Pipe/Pump (FR-34).

Generate New Pipes / Pumps

Use-case name	Generate New Pipes / Pumps
Actors	System (secondary)
Related FRs	FR-35, FR-36
Preconditions	The game session is active. A cistern node exists in the network.

Dialog / Scenario	1. System timer triggers a component-generation event at a predefined interval. 2. System periodically generates new pipes at cisterns (FR-35) and new pumps at cisterns (FR-36). 3. System places the component at a cistern node and marks it as available for pickup. 4. System updates the display to show the new component at the cistern. 5. The component remains at the cistern until collected by a Plumber (Place New Pipe or Insert New Pump).

End Game / Declare Winner

Use-case name	End Game / Declare Winner
Actors	System (secondary)
Related FRs	FR-37, FR-38, FR-39, FR-40
Preconditions	A game session is active. The timer has reached zero.
Dialog / Scenario	1. System ends the game when the timer reaches zero (FR-37). 2. System halts all water flow, player movement, and automated events. 3. The system then compares the two final team scores: the Plumber score, which is the total amount of water delivered to cisterns, and the Saboteur score, which is the total amount of water leaked from the network. 4. If Plumber score > Saboteur score: Plumber team wins. Otherwise: Saboteur team wins (FR-39). 5. System displays the final result screen with both scores and the winning team (FR-40). 6. Players may choose to start a new game or exit.

3.1 Object catalog

3.1.1 System

The **System** object represents one complete game session. It is responsible for initializing the session, coordinating turns, tracking whether the game is running, and ending the session when the timer expires. It also connects the main domain objects such as the teams, players, pipe network, timer, and scoreboard.

Other objects interact with the System to start play, advance turns, query the current phase of the session, and trigger the end-game procedure. The Game does not perform every domain action directly, but it coordinates the objects that do.

3.1.2 Player

A **Player** represents a human-controlled participant moving on the pipe system. A player has a current position and can perform turn-based actions according to the rules of the game. Player is a common concept shared by both Plumber and Saboteur.

Other objects use Player to determine whose turn it is, where the player is standing, and whether an action is allowed from that location. Player itself provides the basic movement-related responsibility, while role-specific actions belong to its specialized forms.

3.1.3 Plumber

A **Plumber** is a specialized Player whose goal is to maximize the amount of water delivered to cisterns. A Plumber can repair broken pumps, repair punctured pipes, change pump direction, connect free pipe ends, and insert new pumps into the network.

Other domain objects use Plumber when an action requires plumber-specific authorization. The class is responsible only for actions that belong uniquely to the plumber role.

3.1.4 Saboteur

A **Saboteur** is a specialized Player whose goal is to maximize water leakage from the network. A Saboteur can change pump direction and puncture pipes in order to reduce successful water delivery.

Other domain objects use Saboteur when verifying sabotage-specific actions. Like Plumber, it extends the common behavior of Player with its own role-specific responsibilities.

3.1.5 PipeNetwork

The **PipeNetwork** represents the full structure of connected pipes and active elements. It stores the current topology of the system and is responsible for managing connections between network elements.

Other objects query the PipeNetwork to validate movement, attach new elements, split pipes, and recalculate structural relationships after a modification.

3.1.6 NetworkElement

A **NetworkElement** is a common abstraction for all objects that are part of the physical water network. It provides the shared concept of connectivity between pipes and active elements such as springs, cisterns, and pumps.

Other objects use NetworkElement when movement, connection, or adjacency must work uniformly across different concrete network objects.

3.1.7 Pipe

A **Pipe** is a network element that transports water between other elements. A pipe may be intact or punctured, and it may also have a free end. If water reaches a free end or a punctured pipe, the water leaks to the desert.

Other objects use Pipe for movement, connection changes, leakage handling, and inserting new pumps by splitting an existing pipe.

3.1.8 Pump

A **Pump** is an active network element that routes water from one selected input pipe to one selected output pipe. It may connect to multiple pipes, but only one input and one output can be active at a time. A pump can malfunction and then stops transferring water until repaired. It also has a limited connection capacity.

Other objects use Pump for routing changes, water transfer checks, breakdown events, repair actions, and connection validation.

3.1.9 Spring

A **Spring** is a source element from which water enters the network. It is responsible for providing water into the connected system during game progression.

Other objects use Spring as the origin of water flow when the network state is evaluated.

3.1.10 Cistern

A **Cistern** is a destination element that receives water delivered through the network. Water that reaches a cistern contributes to the Plumber team's score. The cistern is also the place where new pipes and pumps are generated according to the game rules.

Other objects use Cistern for score updates and for component generation events.

3.1.11 ScoreBoard

The **ScoreBoard** stores and updates the two team scores during the game. It records the amount of water delivered to cisterns for the Plumber team and the amount of leaked water for the Saboteur team. It also provides the final score comparison when the game ends.

Other objects use ScoreBoard whenever water delivery or leakage is recalculated and when the winner must be declared.

3.1.12 GameTimer

The **GameTimer** represents the time limit of the game session. It counts down from the configured duration and triggers the end-game procedure when the remaining time reaches zero.

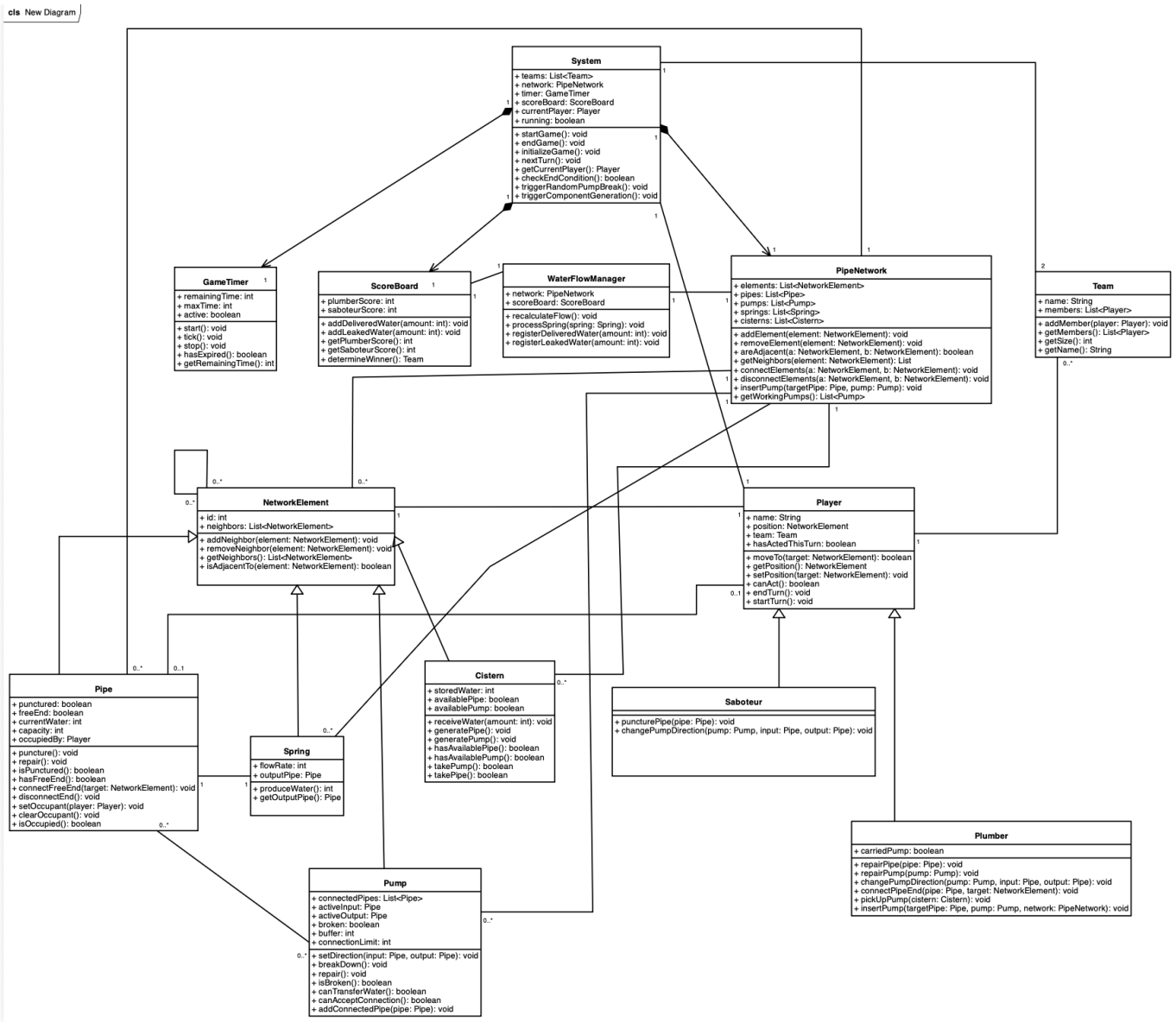
Other objects use GameTimer to query the remaining time and determine whether the session should still accept gameplay actions.

3.1.13 WaterFlowManager

The **WaterFlowManager** is responsible for evaluating how water moves through the current network configuration. It determines whether water is successfully delivered to cisterns, blocked by broken pumps, or leaked through punctured pipes or free pipe ends.

Other objects rely on WaterFlowManager after any structural or state change, such as repairing a pump, changing pump direction, inserting a pump, or puncturing a pipe.

3.2 Static structure diagrams



3.3 Class description

3.3.1 Cistern

Responsibility
The Cistern class represents a destination node in the pipe network. It stores delivered water and periodically makes new pipes and pumps available according to the game rules. It is also the location where plumbers can collect newly generated pumps, and in your revised model it is the origin point of newly generated pipes.

Superclasses
NetworkElement

Interfaces
None

Attributes

- storedWater: int — amount of water delivered to this cistern
- availablePipe: boolean — indicates whether a generated pipe is currently available
- availablePump: boolean — indicates whether a generated pump is currently available

Methods

- receiveWater(amount: int): void — adds delivered water to the cistern
- generatePipe(): void — makes a new pipe available at the cistern
- generatePump(): void — makes a new pump available at the cistern
- hasAvailablePipe(): boolean — returns whether a pipe is available
- hasAvailablePump(): boolean — returns whether a pump is available
- takePump(): boolean — marks the generated pump as collected
- takePipe(): boolean — marks the generated pipe as used or connected

3.3.2 GameTimer

Responsibility
The GameTimer class manages the time limit of the game session. It counts down from the configured duration and indicates when the game must end. Your revised game ending is timer-based, so this class is necessary for the end-game logic.

Superclasses
None

Interfaces
None

Attributes

- remainingTime: int — remaining game time
- maxTime: int — initial configured game duration
- active: boolean — indicates whether the timer is currently running

Methods

- start(): void — starts the timer
- tick(): void — decreases remaining time
- stop(): void — stops the timer
- hasExpired(): boolean — returns true if time reached zero
- getRemainingTime(): int — returns remaining tim

3.3.3 NetworkElement

Responsibility
The NetworkElement class represents the common properties of all physical elements that belong to the pipe system. It provides the shared basis for connectivity, position-based interaction, and adjacency inside the network. Pipes, pumps, springs, and cisterns are all treated as network elements in the game world.

Superclasses
None

Interfaces
None

Attributes

- id: int — unique identifier of the element
- neighbors: List<NetworkElement> — directly connected neighboring elements

Methods

- addNeighbor(element: NetworkElement): void — connects another network element
- removeNeighbor(element: NetworkElement): void — removes a connection
- getNeighbors(): List<NetworkElement> — returns connected neighboring elements
- isAdjacentTo(element: NetworkElement): boolean — checks adjacency

3.3.4 Pipe

Responsibility
The Pipe class represents a water-carrying segment in the network. A pipe may be intact or punctured, and it may also have a free end. Water leaks to the desert when the pipe is punctured or when water reaches a free end. Only one player may occupy a pipe at a time. Pipes can also be split when a plumber inserts a new pump.

Superclasses
NetworkElement

Interfaces
None

Attributes

- punctured: boolean — indicates whether the pipe is leaking

- freeEnd: boolean — indicates whether one end is not connected
- currentWater: int — amount of water currently in the pipe
- capacity: int — maximum amount of water the pipe can hold or transfer
- occupiedBy: Player — player currently standing on the pipe, if any

Methods

- puncture(): void — changes the pipe to leaking state
- repair(): void — restores the pipe to intact state
- isPunctured(): boolean — returns whether the pipe is leaking
- hasFreeEnd(): boolean — returns whether the pipe has an unconnected end
- connectFreeEnd(target: NetworkElement): void — connects the free end to another element
- disconnectEnd(): void — disconnects one end of the pipe
- setOccupant(player: Player): void — sets the player occupying the pipe
- clearOccupant(): void — removes the current occupant
- isOccupied(): boolean — returns whether the pipe is occupied

3.3.5 PipeNetwork

Responsibility
The PipeNetwork class stores the full topology of the pipe system. It manages the collection of pipes and active elements, validates adjacency and connections, and supports structural modifications such as connecting a free pipe end or inserting a pump into a pipe. This class is especially useful because many of your use cases depend on network-level validation.

Superclasses
None

Interfaces
None

Attributes

- elements: List<NetworkElement> — all elements in the network
- pipes: List<Pipe> — all pipes in the network
- pumps: List<Pump> — all pumps in the network
- springs: List<Spring> — all springs in the network
- cisterns: List<Cistern> — all cisterns in the network

Methods

- addElement(element: NetworkElement): void — adds an element to the network
- removeElement(element: NetworkElement): void — removes an element from the network
- areAdjacent(a: NetworkElement, b: NetworkElement): boolean — checks whether two elements are connected
- getNeighbors(element: NetworkElement): List<NetworkElement> — returns neighboring elements
- connectElements(a: NetworkElement, b: NetworkElement): void — creates a connection between two elements

- disconnectElements(a: NetworkElement, b: NetworkElement): void — removes a connection
- insertPump(targetPipe: Pipe, pump: Pump): void — splits a pipe and places a pump between the two resulting segments
- getWorkingPumps(): List<Pump> — returns the pumps currently in working state

3.3.6 Player

Responsibility
The Player class represents a human-controlled participant in the game. It stores the player's identity, current location, and turn-related status. Shared behavior such as movement and turn completion belongs here, while role-specific actions belong to Plumber and Saboteur. This directly supports the professor's expectation that Plumber and Saboteur share a common concept.

Superclasses
None

Interfaces
None

Attributes

- name: String — player name
- position: NetworkElement — current position in the network
- team: Team — team to which the player belongs
- hasActedThisTurn: boolean — indicates whether the player already performed an action this turn

Methods

- moveTo(target: NetworkElement): boolean — attempts to move the player to a connected destination
- getPosition(): NetworkElement — returns current position
- setPosition(target: NetworkElement): void — sets current position
- canAct(): boolean — returns whether the player may still act this turn
- endTurn(): void — marks the player's turn as finished
- startTurn(): void — resets per-turn action status

3.3.7 Plumber

Responsibility
The Plumber class represents a player whose objective is to maximize delivered water. A plumber can repair broken pumps, repair punctured pipes, change pump direction, connect the free end of a generated pipe, collect generated pumps from cisterns, and insert a new pump into an existing pipe. These actions are explicitly supported by the problem definition and your revised FRs.

Superclasses
Player

Interfaces
None

Attributes

- carriedPump: boolean — indicates whether the plumber currently carries a collected pump

Methods

- repairPipe(pipe: Pipe): void — repairs a punctured pipe
- repairPump(pump: Pump): void — repairs a broken pump
- changePumpDirection(pump: Pump, input: Pipe, output: Pipe): void — changes the active routing of a pump
- connectPipeEnd(pipe: Pipe, target: NetworkElement): void — connects the free end of a pipe to an adjacent element
- pickUpPump(cistern: Cistern): void — collects a generated pump from a cistern
- insertPump(targetPipe: Pipe, pump: Pump, network: PipeNetwork): void — inserts a pump into a pipe by splitting it

3.3.8 Pump

Responsibility
The Pump class represents an active routing element in the network. It can have multiple connected pipes, but only one active input pipe and one active output pipe at a time. It may malfunction, and when broken it cannot transfer water. It also has a connection limit and an internal water buffer, both of which come directly from the problem definition and glossary.

Superclasses
NetworkElement

Interfaces
None

Attributes

- connectedPipes: List<Pipe> — pipes attached to this pump
- activeInput: Pipe — currently selected input pipe
- activeOutput: Pipe — currently selected output pipe
- broken: boolean — indicates whether the pump is malfunctioning
- buffer: int — internal temporary water storage
- connectionLimit: int — maximum number of connectable pipes

Methods

- setDirection(input: Pipe, output: Pipe): void — selects active input and output pipes
- breakDown(): void — changes the pump to broken state
- repair(): void — restores the pump to working state
- isBroken(): boolean — returns whether the pump is malfunctioning
- canTransferWater(): boolean — checks whether water may pass through the pump
- canAcceptConnection(): boolean — checks whether another pipe may be connected

- addConnectedPipe(pipe: Pipe): void — attaches a pipe to the pump

3.3.9 Saboteur

Responsibility
The Saboteur class represents a player whose objective is to maximize water leakage. A saboteur can puncture pipes and can also change pump direction to divert flow away from cisterns. These are the role-specific sabotage actions described in the problem definition and use cases.

Superclasses
Player

Interfaces
None

Attributes
No additional public attributes are necessary in the first version.

Methods

- puncturePipe(pipe: Pipe): void — punctures an intact pipe
- changePumpDirection(pump: Pump, input: Pipe, output: Pipe): void — changes the routing of a working pump

3.3.10 ScoreBoard

Responsibility
The ScoreBoard class stores the scores of the two teams during the game. In your revised model, one score represents water delivered to cisterns and the other represents water leaked from the network. It is also responsible for comparing the final values and determining the winner at game end.

Superclasses
None

Interfaces
None

Attributes

- plumberScore: int — amount of water delivered to cisterns
- saboteurScore: int — amount of leaked water

Methods

- addDeliveredWater(amount: int): void — increases the plumber score
- addLeakedWater(amount: int): void — increases the saboteur score
- getPlumberScore(): int — returns plumber score
- getSaboteurScore(): int — returns saboteur score
- determineWinner(): Team — returns the winning team based on final comparison

3.3.11 Spring

Responsibility
The Spring class represents a source node from which water enters the network. It produces water that then flows through pipes and pumps toward cisterns. Springs are part of the initial network and are essential for the water-flow logic of the game.

Superclasses
NetworkElement

Interfaces
None

Attributes

- flowRate: int — amount of water produced during one simulation step
- outputPipe: Pipe — pipe connected to the spring

Methods

- produceWater(): int — generates water for the current simulation step
- getOutputPipe(): Pipe — returns connected output pipe

3.3.12 System

Responsibility
The System class coordinates one full game session. It initializes the game, manages turn order, starts and stops the timer, triggers periodic system events, and ends the game when the timer expires. It does not replace the responsibilities of all other classes, but it coordinates them. This matches your object catalog text and also the “Game Engine” responsibilities from the earlier requirements document.

Superclasses
None

Interfaces
None

Attributes

- teams: List<Team> — the two competing teams
- network: PipeNetwork — the active network
- timer: GameTimer — session timer
- scoreBoard: ScoreBoard — current scores
- currentPlayer: Player — player whose turn is active
- running: boolean — indicates whether the game session is active

Methods

- startGame(): void — starts a new game session
- endGame(): void — terminates the session and triggers winner declaration
- initializeGame(): void — creates the initial network and prepares players

- nextTurn(): void — advances control to the next player
- getCurrentPlayer(): Player — returns the active player
- checkEndCondition(): boolean — checks whether the timer expired
- triggerRandomPumpBreak(): void — randomly breaks a working pump
- triggerComponentGeneration(): void — generates new pipes or pumps at cisterns

3.3.13 Team

Responsibility
The Team class represents one side in the game: the plumber team or the saboteur team. It groups players sharing the same goal and makes it possible to express the winner and score comparison at team level instead of only at player level. Since the problem definition explicitly describes two teams, this class is justified and useful in the analysis model.

Superclasses
None

Interfaces
None

Attributes

- name: String — team name
- members: List<Player> — players belonging to the team

Methods

- addMember(player: Player): void — adds a player to the team
- getMembers(): List<Player> — returns team members
- getSize(): int — returns number of players
- getName(): String — returns team name

3.3.14 WaterFlowManager

Responsibility
The WaterFlowManager class evaluates how water moves through the current network configuration. It determines whether water is delivered to cisterns, blocked by broken pumps, leaked through punctured pipes, or lost through free pipe ends. This is a good responsibility separation because many of your use cases explicitly say that the system recalculates water flow after a change in the network.

Superclasses
None

Interfaces
None

Attributes

- network: PipeNetwork — the network being evaluated
- scoreBoard: ScoreBoard — score holder updated according to flow result

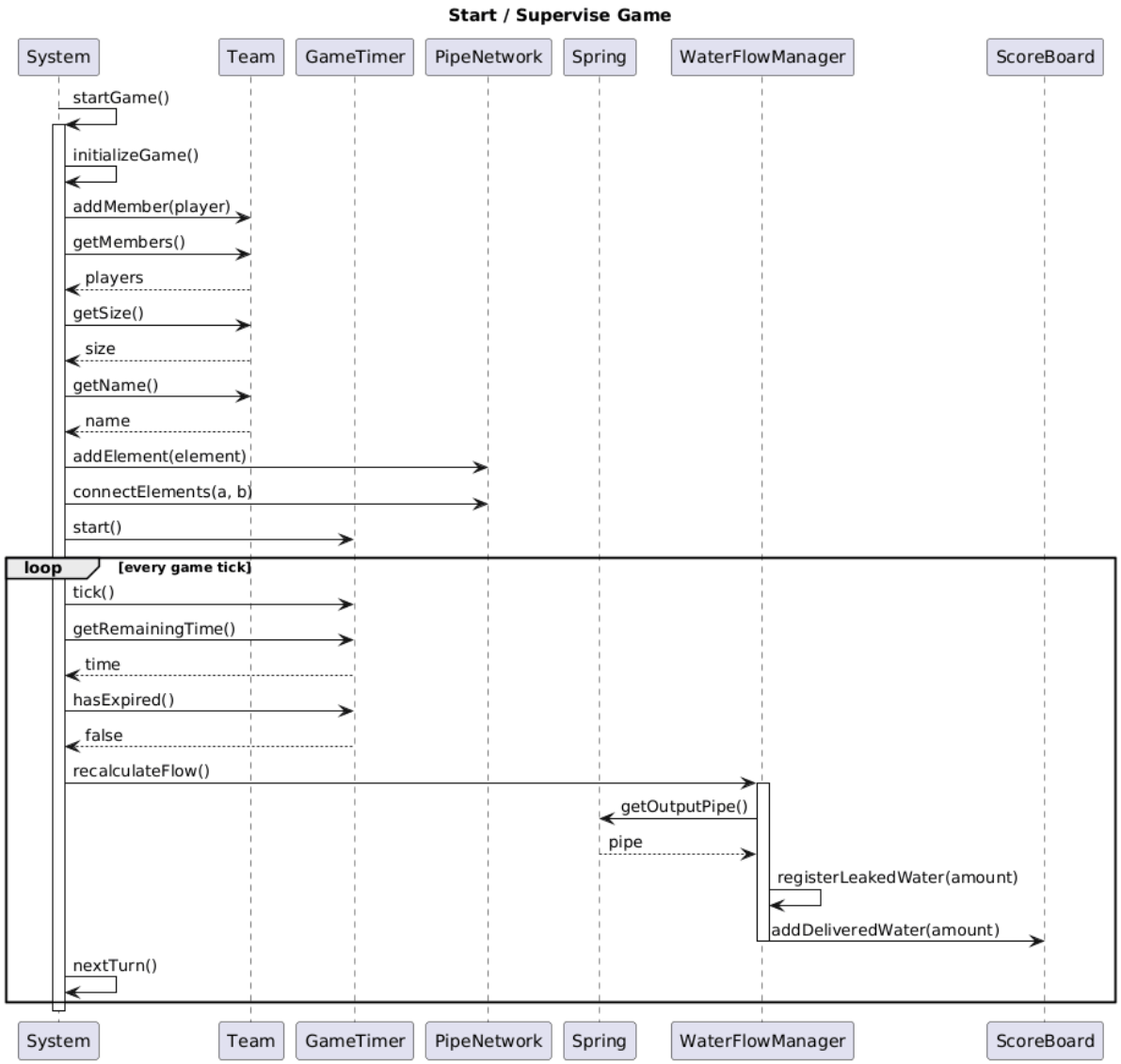
Methods

- recalculateFlow(): void — recalculates water movement in the network
- processSpring(spring: Spring): void — starts flow processing from a spring
- registerDeliveredWater(amount: int): void — records delivered water
- registerLeakedWater(amount: int): void — records leaked water

3.4 Sequence diagrams

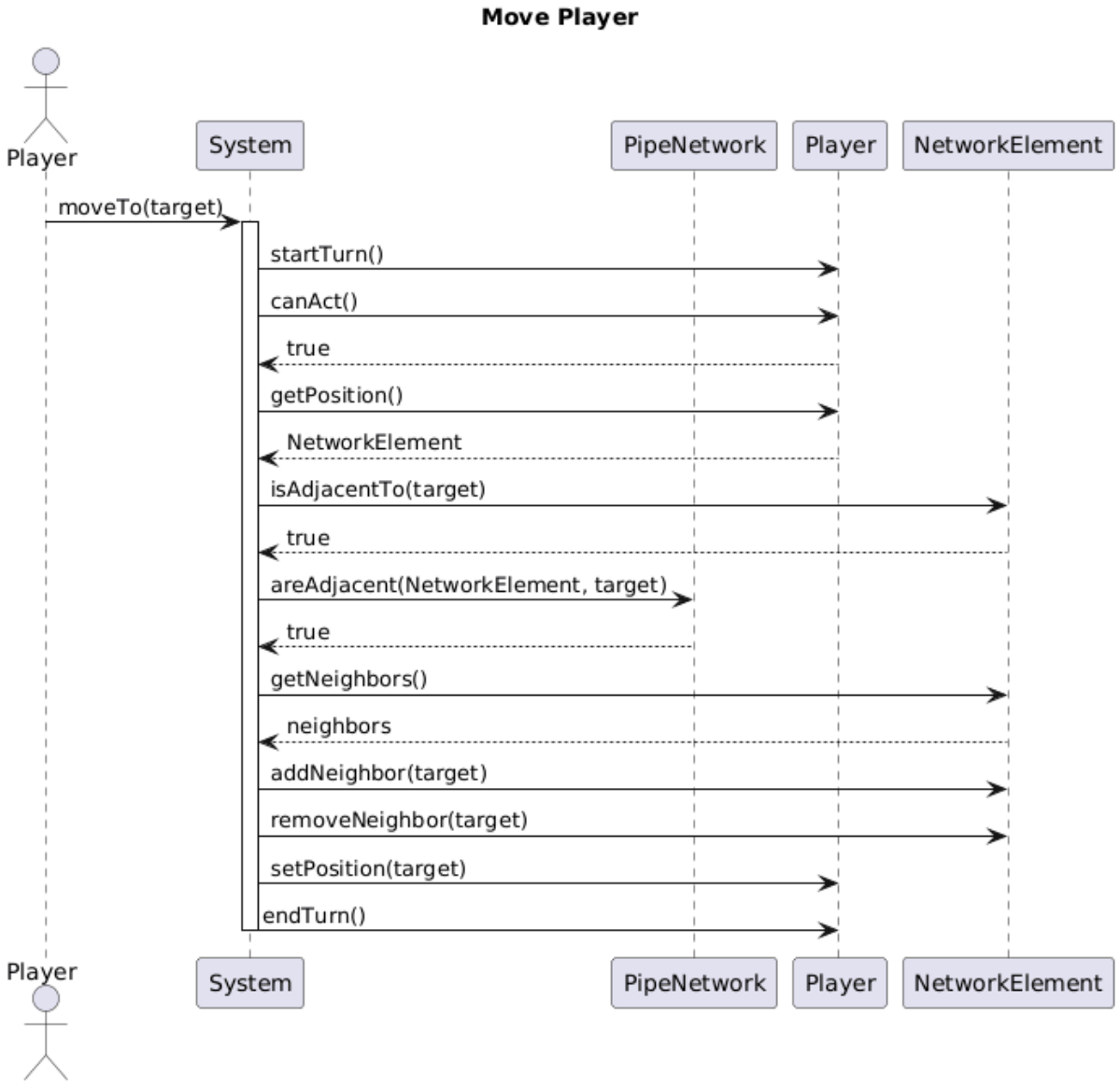
3.4.1 Start / Supervise Game

The System initializes the game by setting up teams, generating the initial pipe network, and starting the timer. It then loops through game ticks, managing water flow, scoring, and player turns until the timer expires.



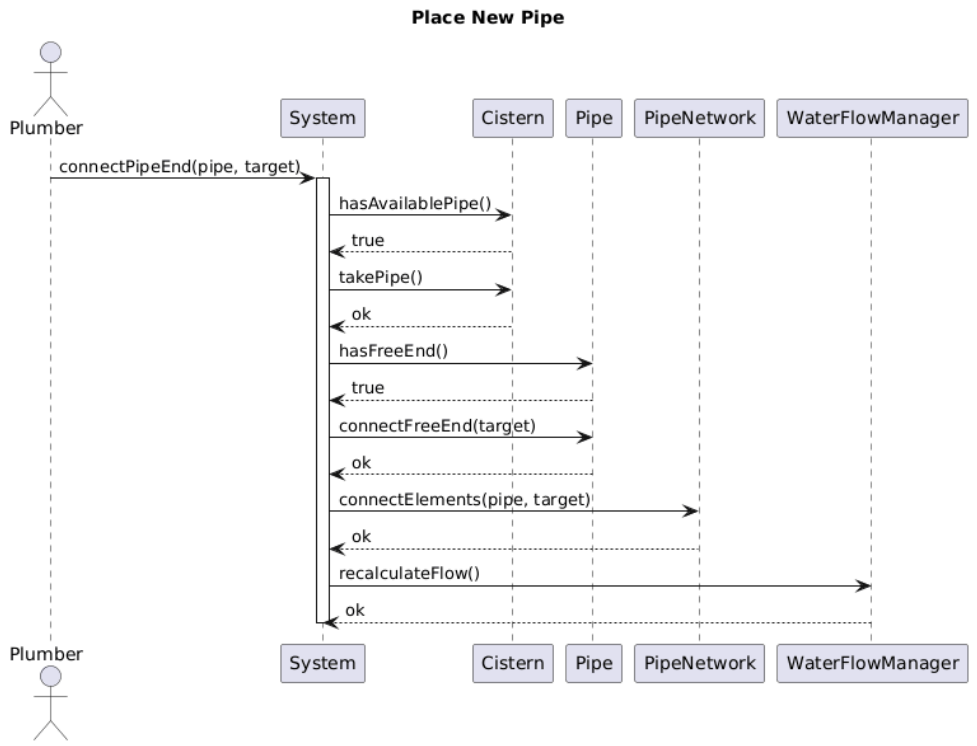
3.4.2 Move Player

A player requests to move to an adjacent network element. The System validates the player's position, checks adjacency, and updates the player's position if the move is valid.



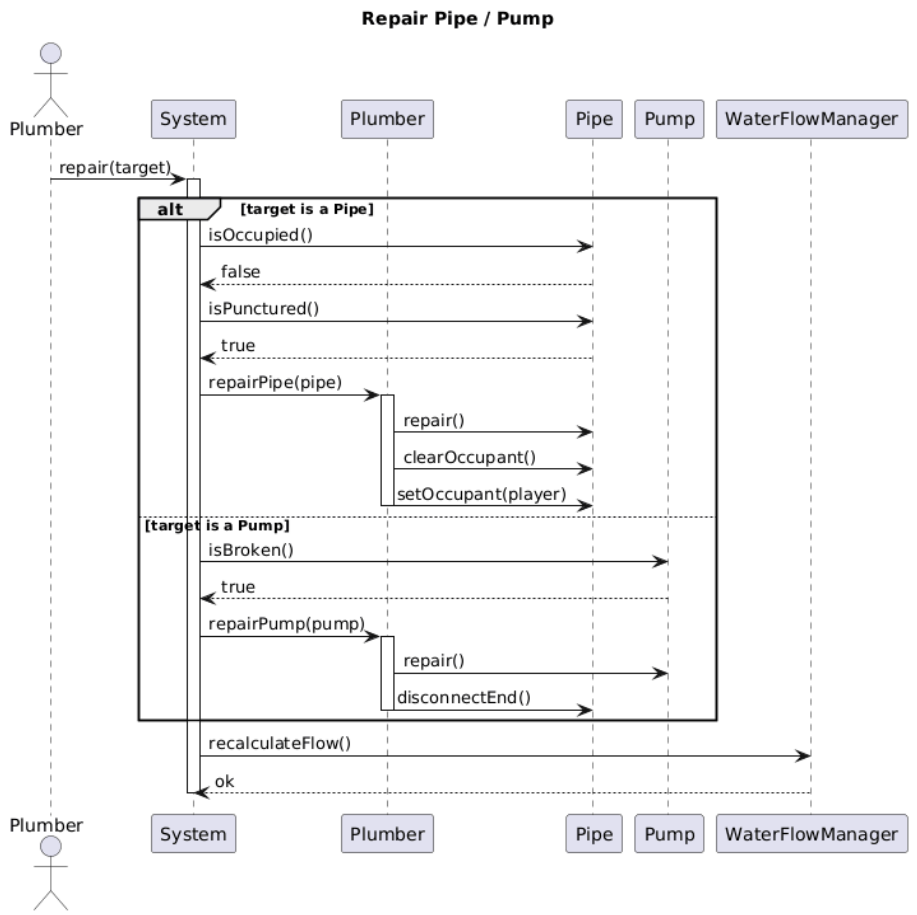
3.4.3 Place New Pipe

A Plumber selects an available pipe at a cistern and connects its free end to a target network element. The System validates the connection and recalculates water flow if the placement is valid.



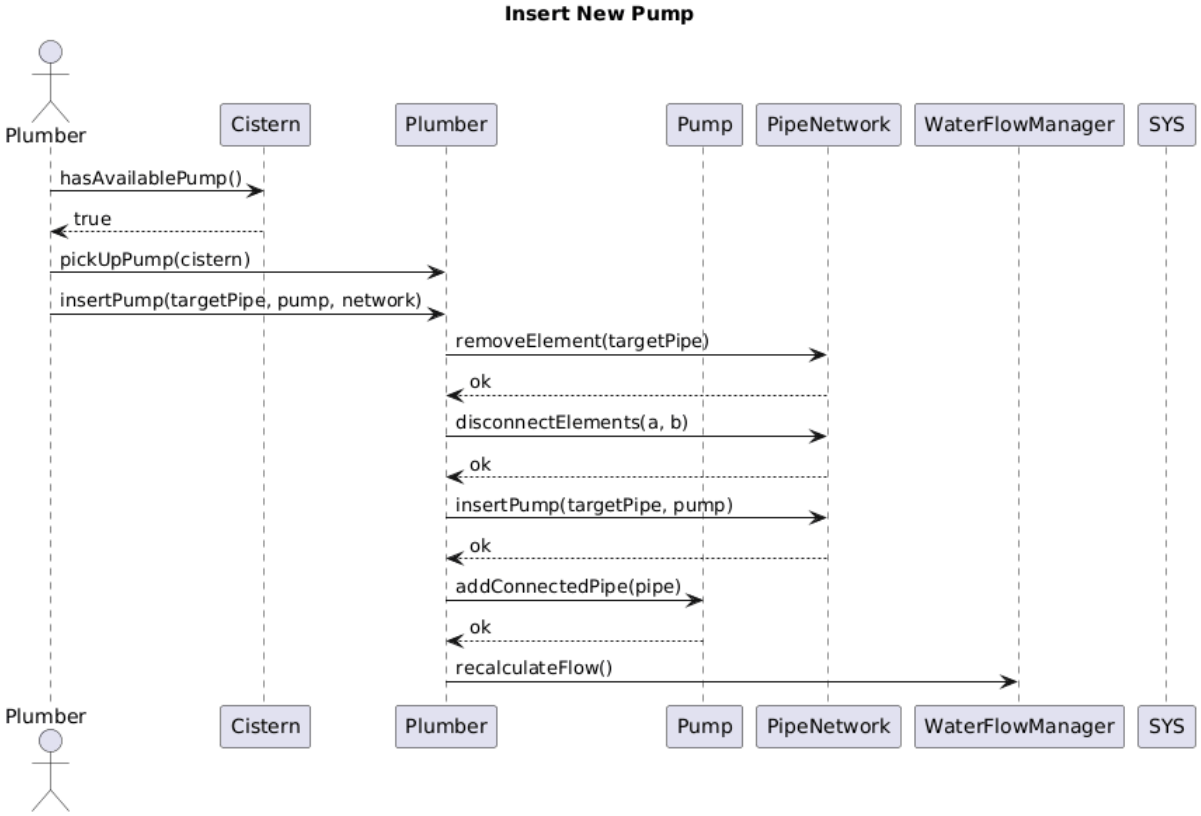
3.4.4 Repair Pipe / Pump

A Plumber repairs either a punctured pipe or a broken pump depending on the target element they are standing on. Water flow is recalculated after the repair is complete.



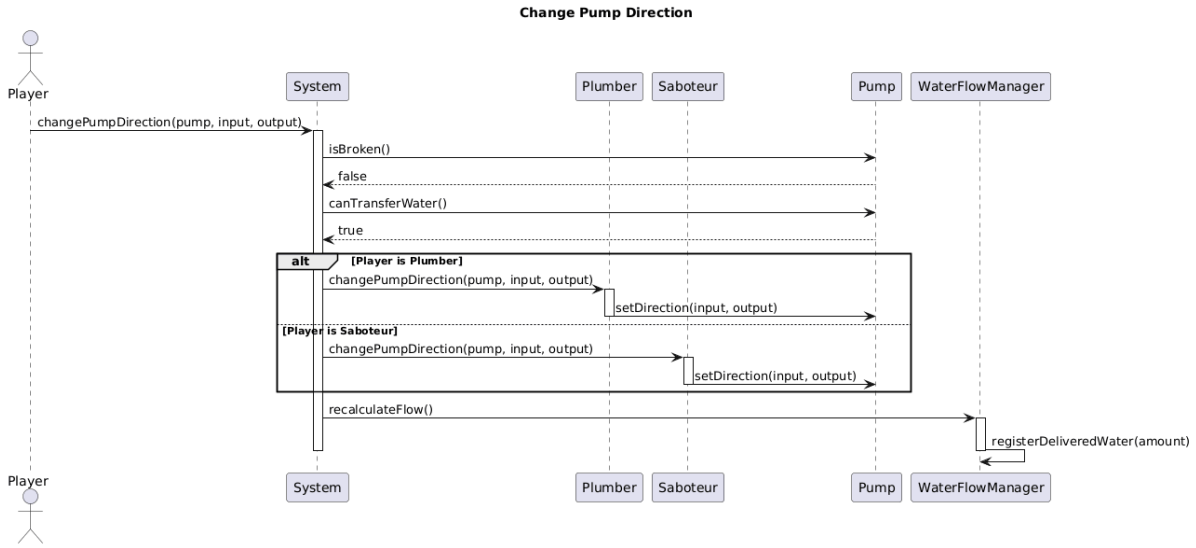
3.4.5 Insert New Pump

A Plumber collects a pump from a cistern and inserts it into an existing pipe by splitting it into two segments. The network is updated and water flow is recalculated.



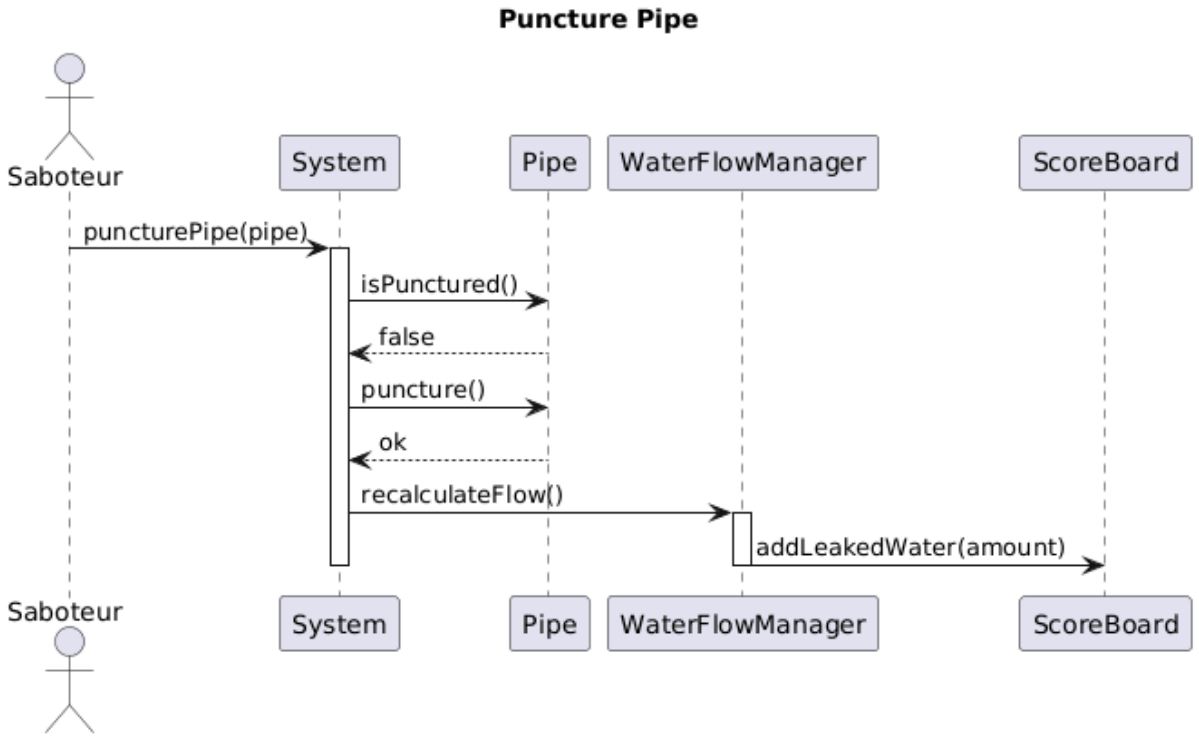
3.4.6 Change Pump Direction

Either a Plumber or Saboteur changes the active input/output routing of a pump they are standing on. The System verifies the pump is working before applying the new direction and recalculating flow.



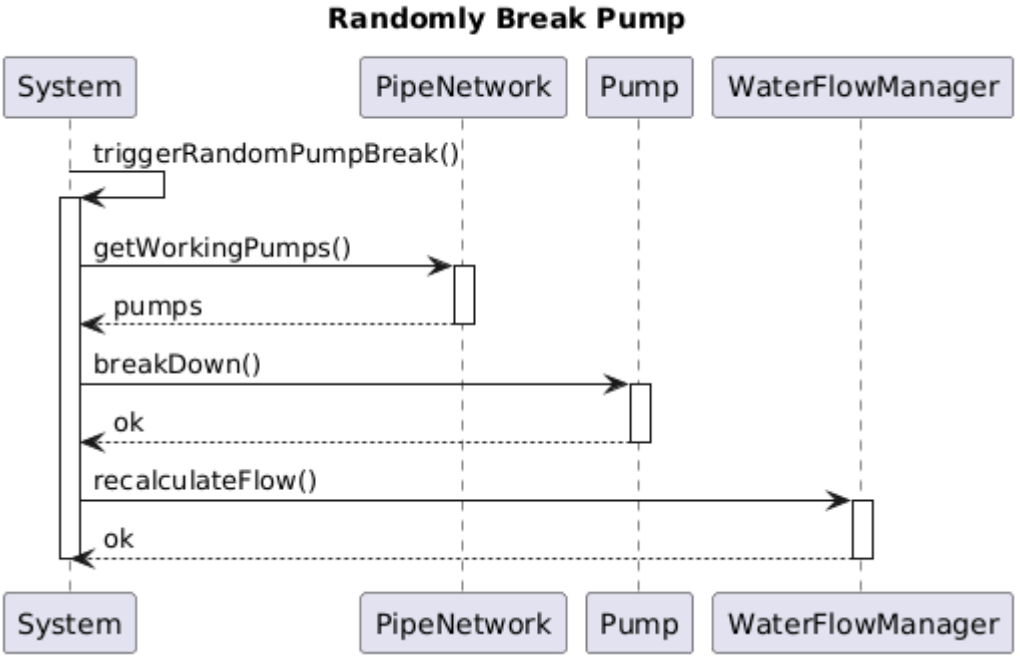
3.4.7 Puncture Pipe

A Saboteur punctures an intact pipe they are standing on, causing water to leak to the desert. The System updates the pipe state and increases the saboteur score accordingly.



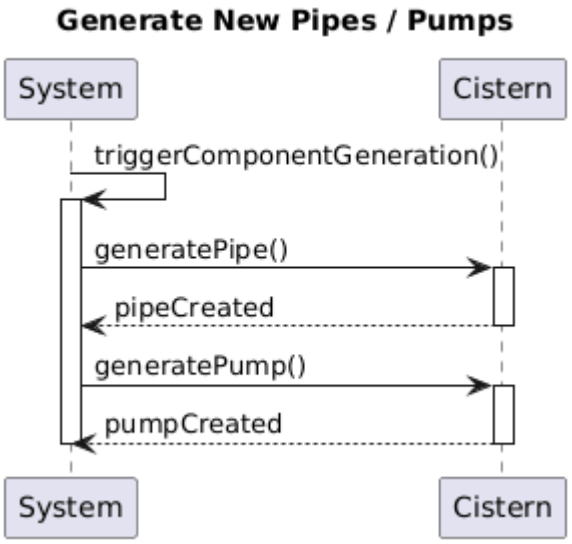
3.4.8 Randomly Break Pump

The System randomly selects a working pump and sets it to a broken state during gameplay. Water flow is recalculated to reflect the pump being out of order.



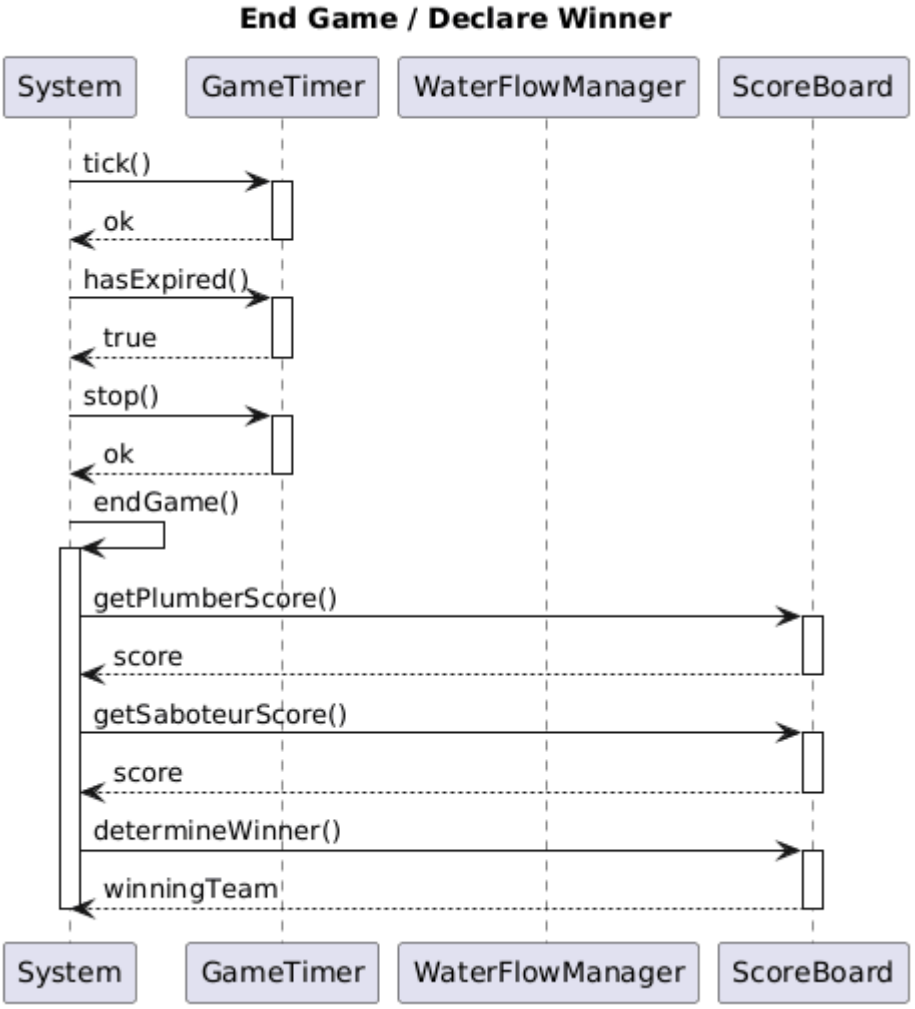
3.4.9 Generate New Pipes / Pumps

The System periodically checks cisterns and generates new pipes or pumps if none are currently available. The new components remain at the cistern until collected by a Plumber.



3.4.10 End Game / Declare Winner

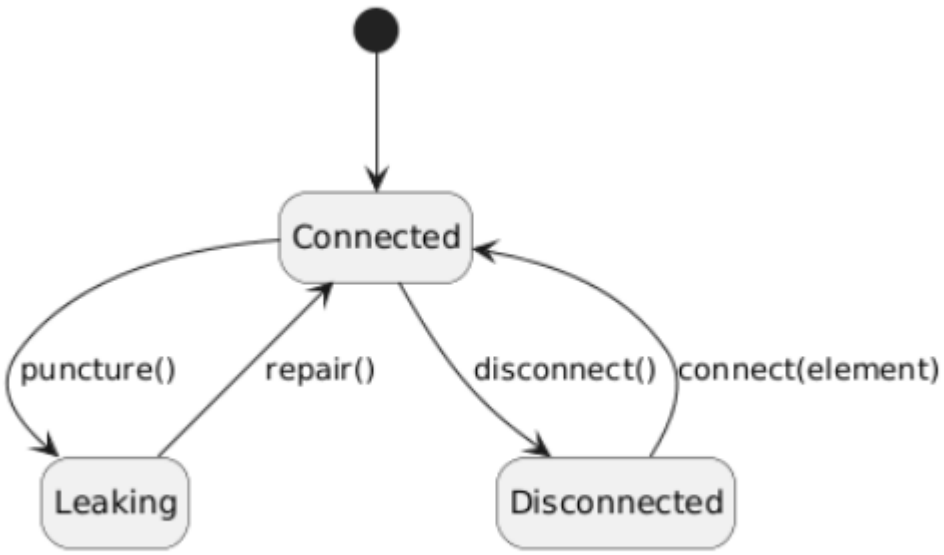
When the timer expires, the System stops all gameplay, compares the total delivered water against the total leaked water, and declares the winning team based on the final scores.


End Game / Declare Winner
System
GameTimer
WaterFlowManager
ScoreBoard
tick()
ok
hasExpired()
true
stop()
ok
endGame()
getPlumberScore()
score
getSaboteurScore()
score
determineWinner()
winningTeam
System
GameTimer
WaterFlowManager
ScoreBoard

3.5 State-charts

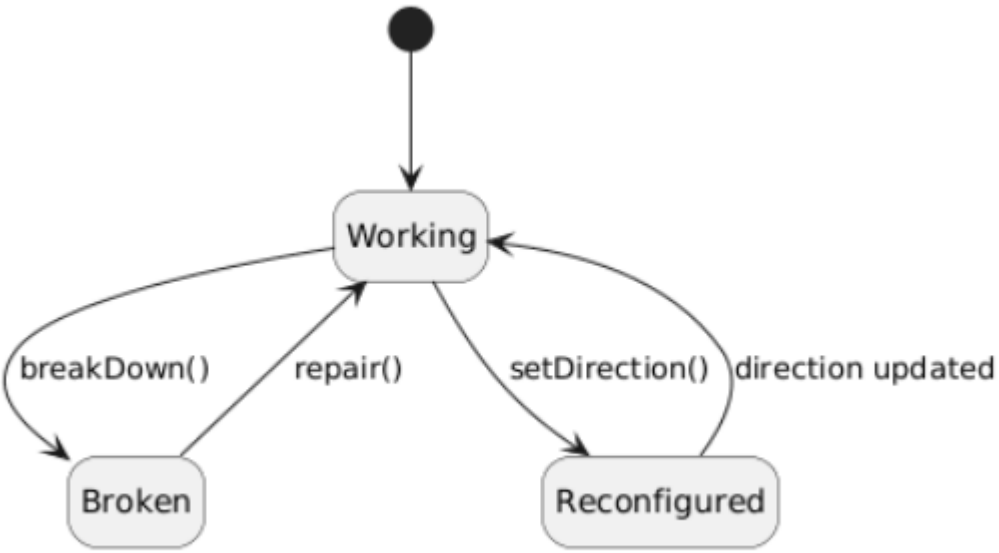
3.5.1 Pipe class

- **Connected**: Both ends attached to network elements.
- **Leaking**: The pipe is punctured and leaks water.
- **Disconnected**: One end is free.



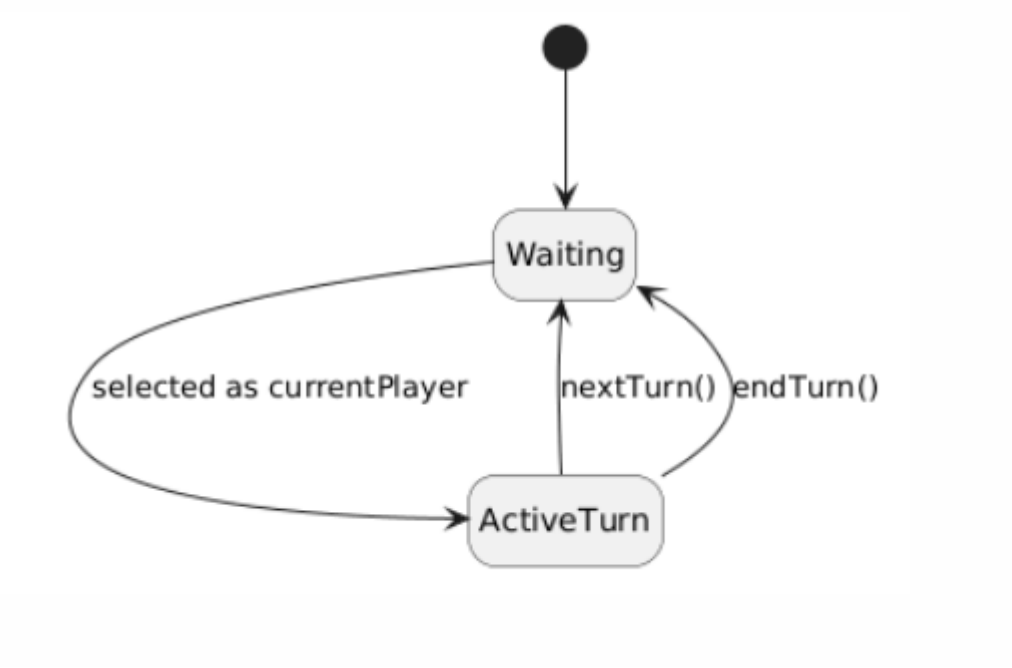
3.5.2 Pump class

- Working: Pump is operational and may transfer water.
- Broken: Pump is malfunctioning and cannot transfer water.
- Reconfigured: Pump direction is being changed.



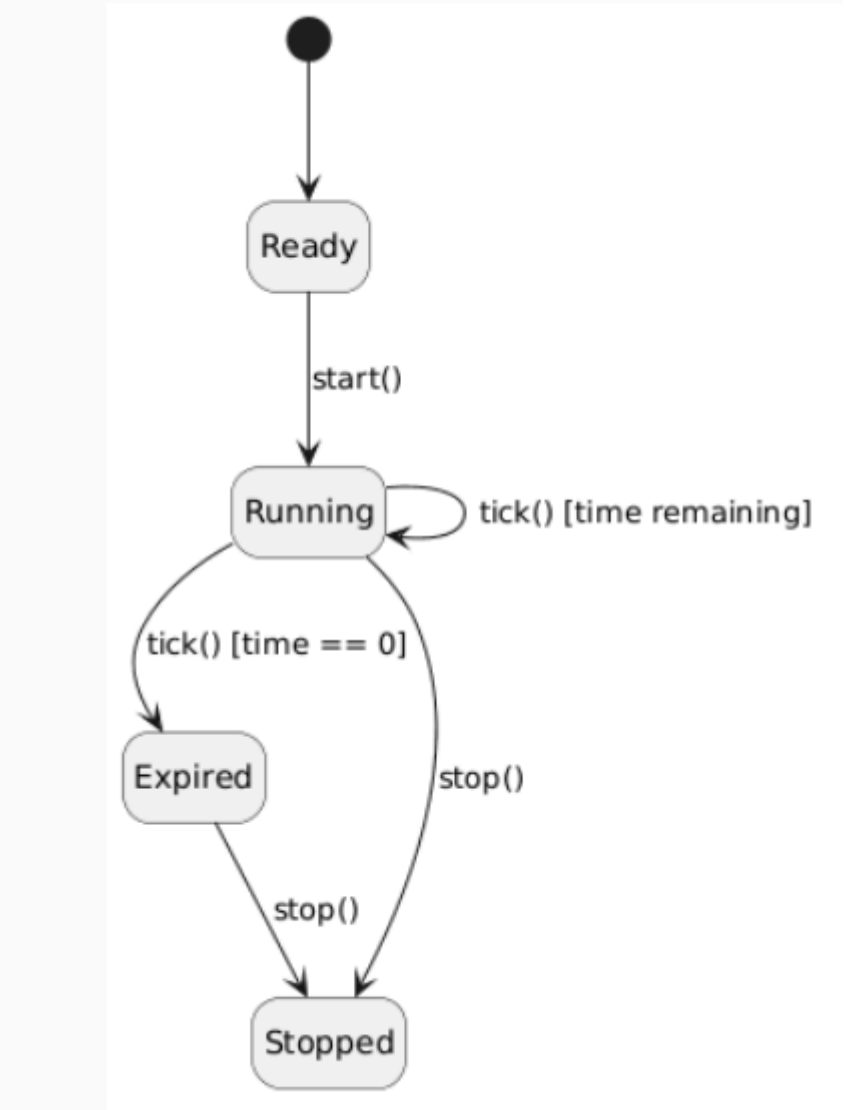
3.5.3 Player class

- **Waiting**: It is not this player's turn.
- **Active**: It is this player's turn and the player may act.
- **TurnFinished**: The player has completed the turn.



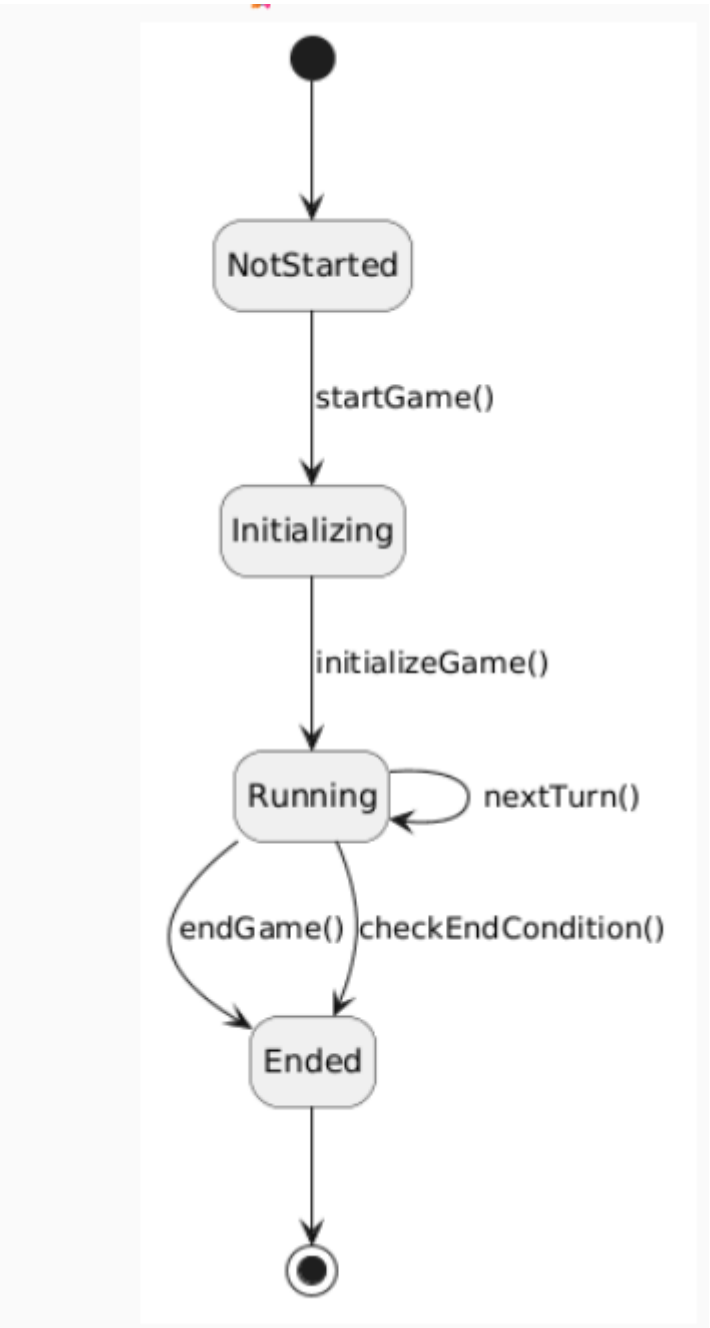
3.5.4 GameTimer class

- **Ready**: Timer is initialized but not running.
- **Running**: Timer is actively counting down.
- **Expired**: Remaining time reached zero.
- **Stopped**: Timer has been stopped.



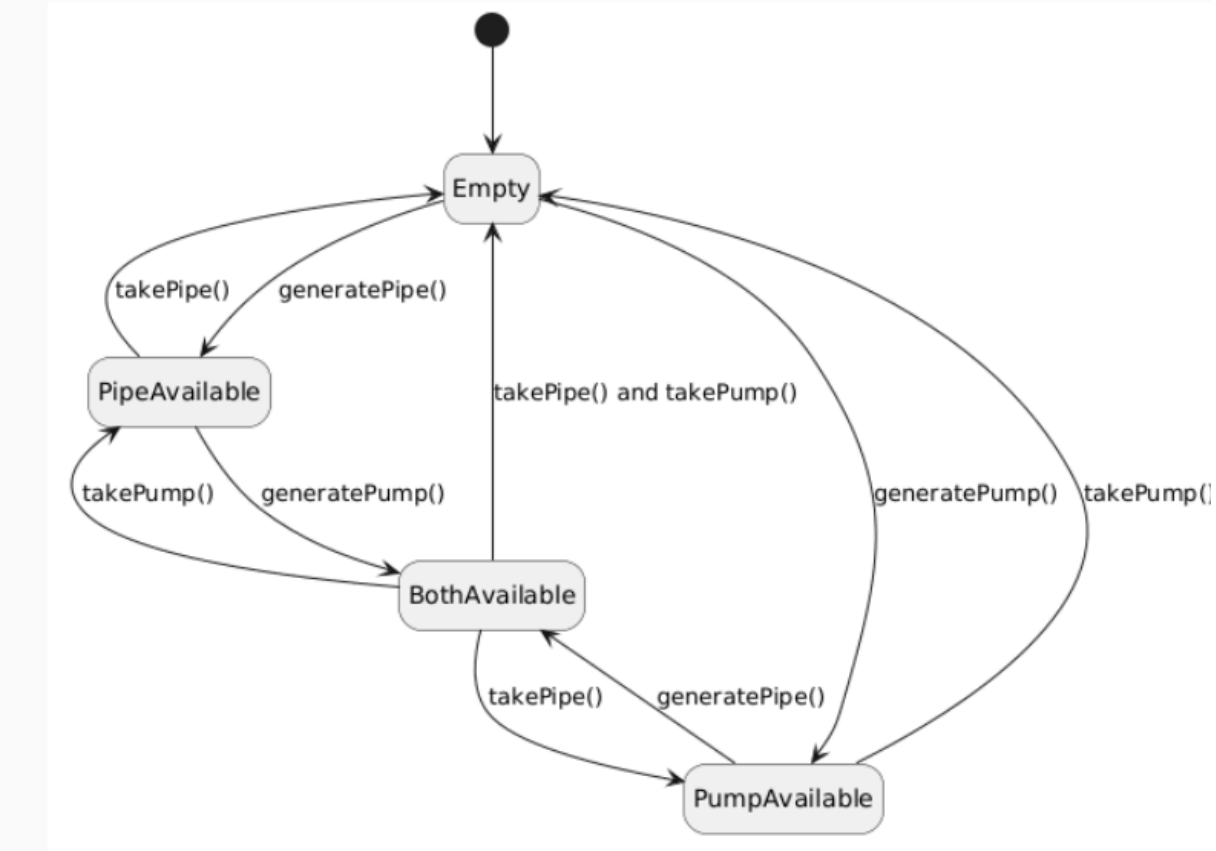
3.5.5 System class

- **NotStarted**: No active session exists.
- **Initializing**: Initial game objects are being prepared.
- **Running**: The game session is active.
- **Ended**: The session has finished.



3.5.6 Cistern class

- **Empty**: No generated component is currently available.
- **PipeAvailable**: A generated pipe is available.
- **PumpAvailable**: A generated pump is available.
- **BothAvailable**: Both a pipe and a pump are available.


Empty
takePipe()
generatePipe()
PipeAvailable
takePipe() and takePump()
takePump()
generatePump()
generatePump()
takePump()
BothAvailable
takePipe()
generatePipe()
PumpAvailable

3.6 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
2026.03.07 11:00	7 hours	Ibrahim, Arda, Ilgın, Hameez, Yahya	Group meeting conducted physically at the Corvinus University Library. The team carefully reviewed the professor's feedback on the previously submitted Requirements Documentation. The feedback was analyzed line by line and discussed in detail. Based on the remarks, the team revised the previous document and corrected several sections. Chapter 0 was added to the new document in order to explicitly address the feedback and document the corrections made to the functional requirements, use cases, and non-functional requirements. The team ensured that the functional requirements now correspond to the use cases and that the start-game and end-game requirements are explicitly defined. Communication practices were also established: the team communicates throughout the week using Microsoft Teams to coordinate tasks and discuss progress, while longer working sessions are conducted during weekend meetings at the library.
2026.03.08 09:00	12 hours	Ibrahim, Arda, Ilgın, Hameez (13:00–21:00), Yahya (13:00–21:00)	Teamwork session at Corvinus University Library focused on preparing the **Analysis Model documentation**. Tasks were divided among members to efficiently produce the diagrams and technical descriptions required for the deliverable. Ibrahim designed and prepared the sequence diagrams describing system interactions. Arda created the activity diagrams illustrating system workflows and player actions. Ilgın developed the class diagram and defined relationships between the main domain objects identified in the object catalog. Hameez and Yahya joined the session at 13:00 and contributed to refining the object catalog, verifying consistency between functional requirements and system operations, and assisting with documentation formatting and diagram validation. The team ensured that the analysis model elements remain consistent with the corrected functional requirements and use cases described in Chapter 0.